ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΣΥΣΤΗΜΑΤΑ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ

Αναφορά για την 2η Ομάδα Ασκήσεων

Μέλη Ομάδας

Κωνσταντόπουλος Γεώργιος (03122104)

Πουλόπουλος Ηλίας (03122160)

ΖΩΓΡΑΦΟΥ ΑΠΡΙΛΙΟΣ 2025

Ασκήσεις Προσομοίωσης

1η Άσκηση

α) Δοκιμάζουμε την αποθήκευση των αριθμών ξεκινώντας από το 255 από τη στιγμή που θέλουμε να είναι αποθηκευμένοι με φθίνουσα σειρά οι αριθμοί έως και το μηδέν.

Για αυτό και παράχθηκε ο παρακάτω κώδικας με το output να φαίνεται στην ακριβώς παρακάτω εικόνα όπου βλέπουμε ότι από την θέση 0900 έως και την 09FF είναι αποθηκευμένοι οι αριθμοί που θέλουμε.

```
IN 10H
MVI A,FFH ;Initialize A with 255
LXI H,0900H ;First Memory Address 0900H

;a)Store 255-0 in memory
LOOPA:
MOV M,A ;Store A (current number) to memory
INX H ;Increase memory address by one
CPI 00H ;Check if we've reached 0000 0000
JNZ LOOPA ;If not continue storing numbers
END
```

08FA	00	08FB	00	08FC	00	08FD	00	08FE	00	08FF	00	0900	FF	0901	FE	0902	FD	0903	FC
0904	FB	0905	FA	0906	F9	0907	F8	0908	F7	0909	F6	090A	F5	090B	F4	090C	F3	090D	F2
090E	F1	090F	F0	0910	EF	0911	EE	0912	ED	0913	EC	0914	EB	0915	EA	0916	E9	0917	E8
0918	E7	0919	E6	091A	E5	091B	E4	091C	E3	091D	E2	091E	E1	091F	E0	0920	DF	0921	DE
0922	DD	0923	DC	0924	DB	0925	DA	0926	D9	0927	D8	0928	D7	0929	D6	092A	D5	092B	D4
092C	D3	092D	D2	092E	D1	092F	D0	0930	CF	0931	CE	0932	CD	0933	CC	0934	CB	0935	CA
0936	C9	0937	C8	0938	C7	0939	C6	093A	C5	093B	C4	093C	C3	093D	C2	093E	C1	093F	C0
0940	BF	0941	BE	0942	BD	0943	BC	0944	BB	0945	BA	0946	B9	0947	B8	0948	B7	0949	B6
094A	B5	094B	B4	094C	B3	094D	B2	094E	B1	094F	B0	0950	AF	0951	AE	0952	AD	0953	AC
0954	AB	0955	AA	0956	A9	0957	A8	0958	A7	0959	A6	095A	A5	095B	A4	095C	A3	095D	A2
095E	A1	095F	A0	0960	9F	0961	9E	0962	9D	0963	9C	0964	9B	0965	9A	0966	99	0967	98
0968	97	0969	96	096A	95	096B	94	096C	93	096D	92	096E	91	096F	90	0970	8F	0971	8E
0972	8D	0973	8C	0974	8B	0975	8A	0976	89	0977	88	0978	87	0979	86	097A	85	097B	84
097C	83	097D	82	097E	81	097F	80	0980	7F	0981	7E	0982	7D	0983	7C	0984	7B	0985	7A
0986	79	0987	78	0988	77	0989	76	098A	75	098B	74	098C	73	098D	72	098E	71	098F	70
0990	6F	0991	6E	0992	6D	0993	6C	0994	6B	0995	6A	0996	69	0997	68	0998	67	0999	66
099A	65	099B	64	099C	63	099D	62	099E	61	099F	60	09A0	5F	09A1	5E	09A2	5D	09A3	5C
09A4	5B	09A5	5A	09A6	59	09A7	58	09A8	57	09A9	56	09AA	55	09AB	54	09AC	53	09AD	52
09AE	51	09AF	50	09B0	4F	09B1	4E	09B2	4D	09B3	4C	09B4	4B	09B5	4A	09B6	49	09B7	48
09B8	47	09B9	46	09BA	45	09BB	44	09BC	43	09BD	42	09BE	41	09BF	40	09C0	3F	09C1	3E
09C2	3D	09C3	3C	09C4	3B	09C5	3A	09C6	39	09C7	38	09C8	37	09C9	36	09CA	35	09CB	34
09CC	33	09CD	32	09CE	31	09CF	30	09D0	2F	09D1	2E	09D2	2D	09D3	2C	09D4	2B	09D5	2A
09D6	29	09D7	28	09D8	27	09D9	26	09DA	25	09DB	24	09DC	23	09DD	22	09DE	21	09DF	20
09E0	1F	09E1	1E	09E2	1D	09E3	1C	09E4	1B	09E5	1A	09E6	19	09E7	18	09E8	17	09E9	16
09EA	15	09EB	14	09EC	13	09ED	12	09EE	11	09EF	10	09F0	0F	09F1	0E	09F2	0D	09F3	0C
09F4	0B	09F5	0A	09F6	09	09F7	08	09F8	07	09F9	06	09FA	05	09FB	04	09FC	03	09FD	02
09FE	01	09FF	00	0A00	00	0A01	00	0A02	00	0A03	00	0A04	00	0A05	00	0A06	00	0A07	00

β) Για το κομμάτι αυτό θα προσθέσουμε στον ήδη υπάρχον κώδικα κάποια επιπλέον flags για να ελέγχουμε τους αριθμούς για μηδενικά όσο τους αποθηκεύουμε στη μνήμη.

Θα χρειαστούμε έναν counter που κάθε φορά που αποθηκεύεται ένας αριθμός θα γίνεται initialize σε 8, ώστε να μετράει πόσες φορές έχουμε κάνει Rotate τα bit του αριθμού για να ελέγξουμε αν καθένα από αυτά είναι μηδενικό. Πρέπει να γίνουν 8 rotations. Θα μπορούσαμε να ελέγχουμε το πότε ο αριθμός επανέρχεται στην αρχική του μορφή αλλά αυτό δημιουργεί προβλήματα με αροιθμούς που είναι παλίνδρομα με χαρακτηριστικό το 0000 0000. Με το πρώτο rotation σε έναν τέτοιο αριθμό θα έχανε πολλά μηδενικά από την μέτρηση.

```
[...]
LXI D,0008H ;Initialize D as 8, used for count of zeros.
;Initialized as 8 because of 0000 0000.
;When we reach 0000 0000 we have to finish and can't go back to do ZEROCOUNT

LOOPA:
MOV M,A
MOV B,A ;Copy A (current number) as Backup
MVI A,08H ;Counter of 8 bits. We ensure that we check all 8 bits of each
number

;b)Count zeros in each number (store in DE)
ZEROCHECK:
CPI 00H ;Have we checked all 8 bits of the number?
JZ CONT ;Then continue to the next number
DCR A
MOV C,A ;Temporarily move the counter to C
MOV A,B ;Retrieve the current number from B (Backup)
RRC ;Rotate to get one bit out of the number
MOV B,A ;Copy the current number back to B for Backup
MOV A,C ;Retrieve the counter
JNC ZEROCOUNT ;If from RRC we've got a 0 increase D which counts zeros
JMP ZEROCHECK ;Go to the next bit to check

CONT:
MOV A,B ;Retrieve the current number from B (Backup)
INX H ;Increase memory address by one
DCR A ;Decrease A by 1
CPI 00H ;Check if we've reached 0000 0000
JNZ LOOPA ;If not continue storing numbers
```

```
ZEROCOUNT:
INX D
JMP ZEROCHECK

END
```

Επιπλέον, όπως θα δούμε στη συνέχεια, τον D τον αποθηκεύουμε έπειτα σε μία θέση μνήμης για να μην τον χάσουμε μετά το πέρας της εκτέλεσης του προγράμματος. Το αποτέλεσμα που πήραμε ήταν **D: 04, E: 00.** Αυτό σημαίνει ότι οι αριθμοί 255-0 έχουν **0400_{HEX} = $1024_{DEC.}$** που αποτελεί και το σωστό αποτέλεσμα.

γ) Επειδή ο 8085 εκτελεί τις περισσότερες εντολές που χρειαζόμαστε στον συσσωρευτή A και οι υπόλοιποι είναι ήδη κατειλημμένοι (D,E από το ερώτημα (β) και C από το αποτέλεσμα αυτού του ερωτήματος) δεν μπορούμε να ελέγξουμε ταυτόχρονα με την εγγραφή και το πλήθος των αριθμών μεταξύ 10H και 80H. Για αυτό θα χρειαστεί να προσπελάσουμε όλη τη μνήμη από την αρχή και να βρούμε αυτούς τους αριθμούς και να αυξάνουμε τον counter C.

Στο προηγούμενο κομμάτι κώδικα θα προσθέσουμε τα παρακάτω:

```
[...]
JMP INBETWEEN

ZEROCOUNT:
INX D
JMP ZEROCHECK

;c)Get the amount of numbers between 10H and 80H in C
INBETWEEN:
LXI H,0900H ;We're going through the whole memory to search
MVI C,00H ;Initialise counter C

;At the beginning of the memory there is 255 which is surely more than 80H
MORETHAN80:
MOV A,M ;Store the current number of the memory in A
CPI 80H ;Compare it with 80H
INX H ;Next memory address
JZ INCREASE ;If we've reached 80H increase C
JNC MORETHAN80 ;If it's still greater than 80H continue in the loop
CPI 10H ;Compare current number with 10H
JC END ;If it's less than 10H it moves the end, because we don't need to
check the rest
;If it's equal to 10H it will just continue to INCREASE
```

```
INCREASE:
INR C ;Increase the counter of number between 10H and 80H
JMP MORETHAN80

END:
LXI H,0A02H ;Store C in memory so that we don't lose it when it finishes
running
MOV M,C
END
```

Το αποτέλεσμα που πήραμε ήταν **C: 71.** Αυτό σημαίνει ότι οι αριθμοί μεταξύ του 10H και 80H είναι **71HEX = 113DEC.** που αποτελεί και το σωστό αποτέλεσμα.

Ολοκληρωμένο το πρόγραμμα θα βρεθεί στο αρχείο *ex2.1.8085*. Το ολοκληρωμένο output του προγράμματος είναι το παρακάτω:

08FA	00	08FB	00	08FC	00	08FD	00	08FE	00	08FF	00	0900	FF	0901	FE	0902	FD	0903	FC
0904	FB	0905	FA	0906	F9	0907	F8	0908	F7	0909	F6	090A	F5	090B	F4	090C	F3	090D	F2
090E	F1	090F	F0	0910	EF	0911	EE	0912	ED	0913	EC	0914	EB	0915	EA	0916	E9	0917	E8
0918	E7	0919	E6	091A	E5	091B	E4	091C	E3	091D	E2	091E	E1	091F	E0	0920	DF	0921	DE
0922	DD	0923	DC	0924	DB	0925	DA	0926	D9	0927	D8	0928	D7	0929	D6	092A	D5	092B	D4
092C	D3	092D	D2	092E	D1	092F	D0	0930	CF	0931	CE	0932	CD	0933	CC	0934	CB	0935	CA
0936	C9	0937	C8	0938	C7	0939	C6	093A	C5	093B	C4	093C	C3	093D	C2	093E	C1	093F	C0
0940	BF	0941	BE	0942	BD	0943	BC	0944	BB	0945	BA	0946	B9	0947	B8	0948	B7	0949	B6
094A	B5	094B	B4	094C	B3	094D	B2	094E	B1	094F	B0	0950	AF	0951	AE	0952	AD	0953	AC
0954	AB	0955	AA	0956	A9	0957	A8	0958	A7	0959	A6	095A	A5	095B	A4	095C	A3	095D	A2
095E	A1	095F	A0	0960	9F	0961	9E	0962	9D	0963	9C	0964	9B	0965	9A	0966	99	0967	98
0968	97	0969	96	096A	95	096B	94	096C	93	096D	92	096E	91	096F	90	0970	8F	0971	8E
0972	8D	0973	8C	0974	8B	0975	8A	0976	89	0977	88	0978	87	0979	86	097A	85	097B	84
097C	83	097D	82	097E	81	097F	80	0980	7F	0981	7E	0982	7D	0983	7C	0984	7B	0985	7A
0986	79	0987	78	0988	77	0989	76	098A	75	098B	74	098C	73	098D	72	098E	71	098F	70
0990	6F	0991	6E	0992	6D	0993	6C	0994	6B	0995	6A	0996	69	0997	68	0998	67	0999	66
099A	65	099B	64	099C	63	099D	62	099E	61	099F	60	09A0	5F	09A1	5E	09A2	5D	09A3	5C
09A4	5B	09A5	5A	09A6	59	09A7	58	09A8	57	09A9	56	09AA	55	09AB	54	09AC	53	09AD	52
09AE	51	09AF	50	09B0	4F	09B1	4E	09B2	4D	09B3	4C	09B4	4B	09B5	4A	09B6	49	09B7	48
09B8	47	09B9	46	09BA	45	09BB	44	09BC	43	09BD	42	09BE	41	09BF	40	09C0	3F	09C1	3E
09C2	3D	09C3	3C	09C4	3B	09C5	3A	09C6	39	09C7	38	09C8	37	09C9	36	09CA	35	09CB	34
09CC	33	09CD	32	09CE	31	09CF	30	09D0	2F	09D1	2E	09D2	2D	09D3	2C	09D4	2B	09D5	2A
09D6	29	09D7	28	09D8	27	09D9	26	09DA	25	09DB	24	09DC	23	09DD	22	09DE	21	09DF	20
09E0	1F	09E1	1E	09E2	1D	09E3	1C	09E4	1B	09E5	1A	09E6	19	09E7	18	09E8	17	09E9	16
09EA	15	09EB	14	09EC	13	09ED	12	09EE	11	09EF	10	09F0	0F	09F1	0E	09F2	0D	09F3	0C
09F4	0B	09F5	0A	09F6	09	09F7	08	09F8	07	09F9	06	09FA	05	09FB	04	09FC	03	09FD	02
09FE	01	09FF	00	0A00	04	0A01	00	0A02	71	0A03	00	0A04	00	0A05	00	0A06	00	0A07	00

2η Άσκηση

Πρώτα γράφουμε το κομμάτι του κώδικα που εντοπίζει την ακολουθία OFF-ON-OFF

```
START:
MVI A,FFH ;LEDs OFF
STA 3000H
CALL CHECKLSB
JC START ;It exits this loop only when LSB=OFF

STEP1:
CALL CHECKLSB
JNC STEP1 ;It exits this loop only when LSB=ON

STEP2:
CALL CHECKLSB
JC STEP2 ;It continues only when LSB=OFF

CHECKLSB:
LDA 2000H ;Input from dip switch
RRC ;Get LSB
RET
```

Στη συνέχεια γράφουμε τις ρουτίνες που ανάβουν και σβήνουν όλα τα LED:

```
MVI D,4BH ;75 cycles (15/0.2 s)

FLASH:
CALL LEDON
CALL DELAY
CALL LEDOFF
CALL DELAY
CALL CHECKLSB
JC TRIGERRED
DCR D
JZ START
JMP FLASH

LEDON:
MVI A,00H
STA 3000H
RET
```

```
LEDOFF:
MVI A,FFH
STA 3000H
RET
```

Έπειτα τις ρουτίνες που είναι απαραίτητες για τον εντοπισμό του ON κατά τη διάρκεια του FLASH και την ανανέωση του μετρητή D για την χρονοκαθυστέρηση που ανανεώνεται ξανά στα 15s:

```
TRIGERRED:
CALL LEDON
CALL DELAY
CALL LEDOFF
CALL DELAY
CALL CHECKLSB
JNC REFILL
DCR D
JZ START
JMP TRIGERRED

REFILL:
MVI D,4BH
JMP FLASH
```

Η ρουτίνα χρονοκαθυστέρησης είναι η παρακάτω (με αύξηση του E αυξάνεται η καθυστέρηση αν δούμε ότι γίνεται πολύ γρήγορα το FLASH):

```
DELAY:
MVI E,15H
DELAY1:
MVI C,FFH
DELAY2:
DCR C
JNZ DELAY2
DCR E
JNZ DELAY1
RET
```

Για την εκτέλεση του προγράμματος και για να διαρκεί το FLASH 15s χρειάστηκε η μπάρα προσθήκης τεχνητής χρονοκαθυστέρησης να είναι πιο δεξιά από τη μέση.

Το ολοκληρωμένο πρόγραμμα μπορεί να βρεθεί με την ονομασία ***ex2.2.8085*** και να εκτελεστεί.

3η Άσκηση

Οι κώδικες βρίσκονται στα αντίστοιχα αρχεία (***ex2.3.1, ex2.3.2, ex2.3.3)***, λόγω του πολύ μεγάλου μεγέθους τους.

4η Άσκηση

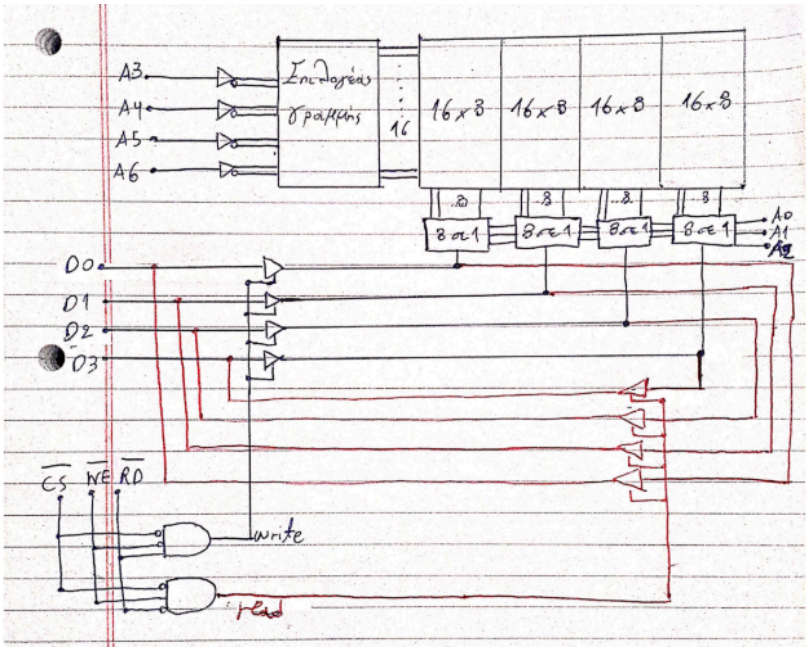
Όπως και στην 3, μπορείτε να βρείτε τον κώδικα με τα σχόλια στο αρχείο ***ex2.4.8085***

Θεωρητικές Ασκήσεις

5η Άσκηση

Θα χωρίσουμε τη μνήμη σε **4 τμήματα των 128 bits** που πρέπει να έχουν 2 διαστάσεις (στη συγκεκριμένη περίπτωση επιλέγουμε **16x8**). Για να επιλέξουμε **γραμμή** χρειαζόμαστε **$\log_2 16$ = 4 bit διεύθυνσης** και για να επιλέξουμε **στήλη** χρειαζόμαστε **$\log_2 8$ = 3 bit διεύθυνσης.**

Όπως και στο παράδειγμα SRAM 1Kx4bits που υπάρχει στις διαφάνειες σχεδιάστηκε η παρακάτω SRAM:



Παράδειγμα: *WRITE 1101 @ 1100 001*

Στη περίπτωση του Write έχουμε:

- CS=1
- WE=1
- RD=0

,που σημαίνει ότι write=1 και read=0.

Ενεργοποιούνται οι μπλε buffers, οι πολυπλέκτες-αποπλέκτες βρίσκονται σε λειτουργία αποπλέκτη καθώς λαμβάνει την είσοδο από τα buffers και τα κάνει πολλές εξόδους (8) προς τη μνήμη για να τα κάνει εγγραφή εκεί.

Τα A0 έως A6 παίρνουν την διεύθυνση στην οποία θα κάνουμε εγγραφή.

Τα A0 έως A2 περιέχουν την στήλη (στη προκειμένη περίπτωση **001=1**) που είναι τα 3 τελευταία bits της διεύθυνσης.

Τα A3 έως A6 περιέχουν την σειρά στην οποία θα γίνει η εγγραφή (**1100=12***)*

Τα D0 έως D3 περιέχουν τα δεδομένα που θα κάνουμε εγγραφή **1101** και το καθένα από αυτά θα γραφεί σε ένα διαφορετικό κομμάτι 16x8 της μνήμης στη θέση (12,1).

Παράδειγμα: *READ 0110 111*

Στη περίπτωση του Read έχουμε:

- CS=1
- WE=0
- RD=1

,που σημαίνει ότι write=1 και read=0.

Αυτή τη φορά ενεργοποιούνται οι κόκκινοι buffers, οι πολυπλέκτες-αποπλέκτες έχουν λειτουργία πολυπλέκτη καθώς από πολλές εισόδους (8) θέλουμε να παίρνουμε έξοδο το 1 bit που θα αποθηκεύεται στα D0 έως D3 και θα παίρνουμε από αυτά το output.

Όμοια με προηγουμένως από τα A0 έως A2 παίρνουμε τη στήλη στην οποία θα αναζητήσουμε το περιεχόμενο της μνήμης, ενώ από τα A3 έως A6 τη σειρά. Για αυτό και η θέση που θα ψάξουμε στο συγκεκριμένο παράδειγμα σε κάθε ένα κομμάτι 16x8 είναι η σειρά **0110=6** και η στήλη **111=7**, δηλαδή η θέση (6,7).

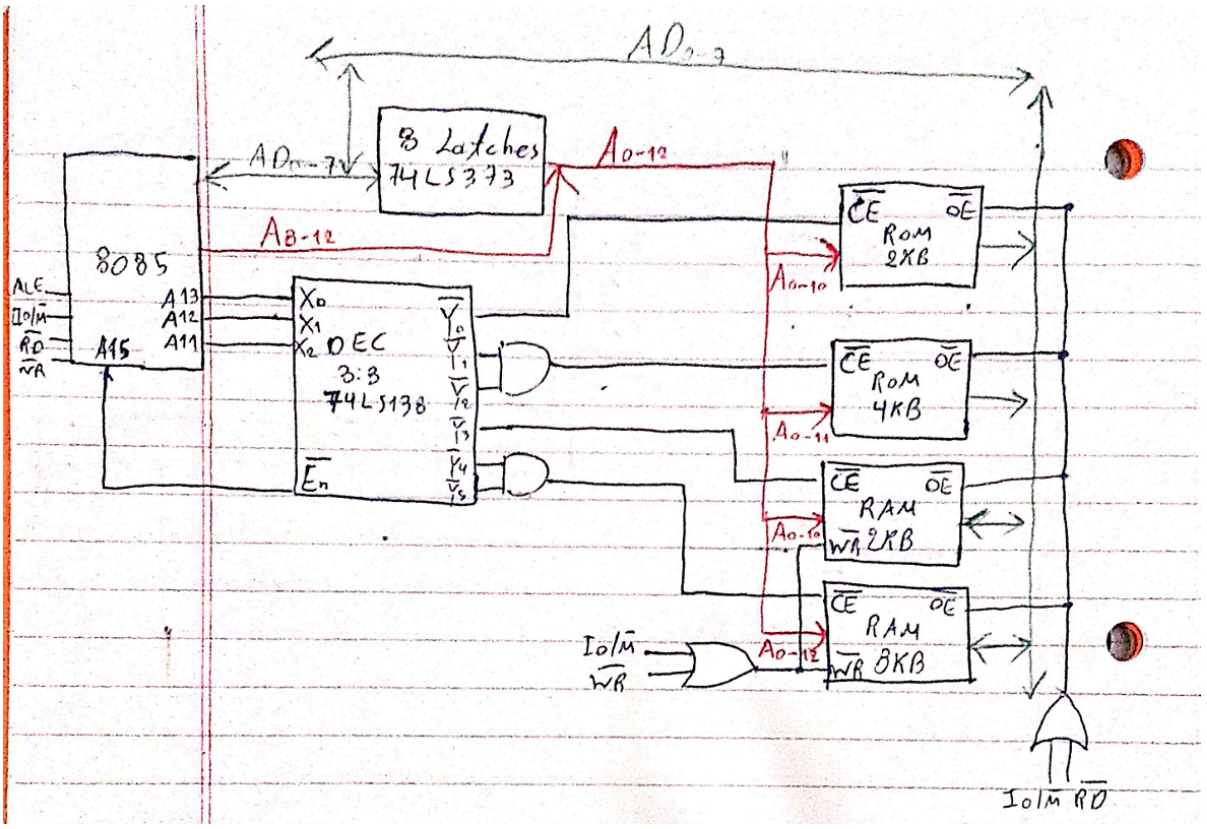
6η Άσκηση

- Στη ROM των 2KB (2^**11**) απαιτούνται **11** bit για τη διευθυνσιοδότηση
- Στη ROM των 4KB (2^**12**) απαιτούνται **12** bit για τη διευθυνσιοδότηση
- Στη RAM των 2KB (2^**11**) απαιτούνται **11** bit για τη διευθυνσιοδότηση
- Στη RAM των 8KB (2^**13**) απαιτούνται **13** bit για τη διευθυνσιοδότηση

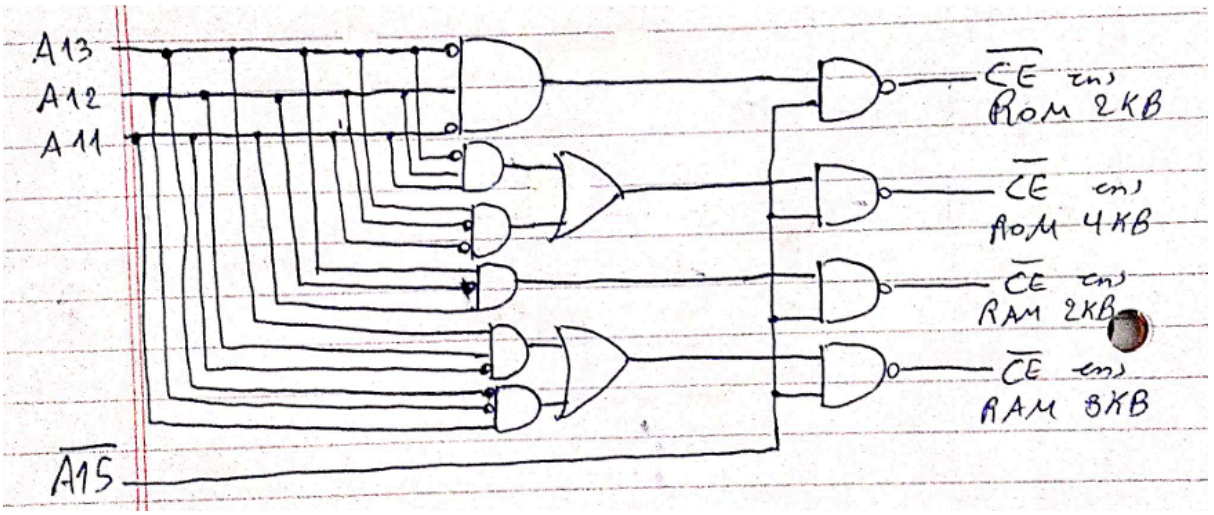
Y	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Address	Memory
Y0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000H	ROM 2KB
	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	17FFH	
Y1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1800H	ROM 4KB
Y2	0	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	27FFH	
Y3	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	2800H	RAM 2KB
	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	2FFFH	
Y4	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	3000H	RAM 8KB
Y5	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	4FFFH	

Βάσει του παραπάνω χάρτη μνήμης σχεδιάσαμε τα παρακάτω συστήματα μνήμης:

α) Χρήσει αποκωδικοποιητή 3 σε 8.

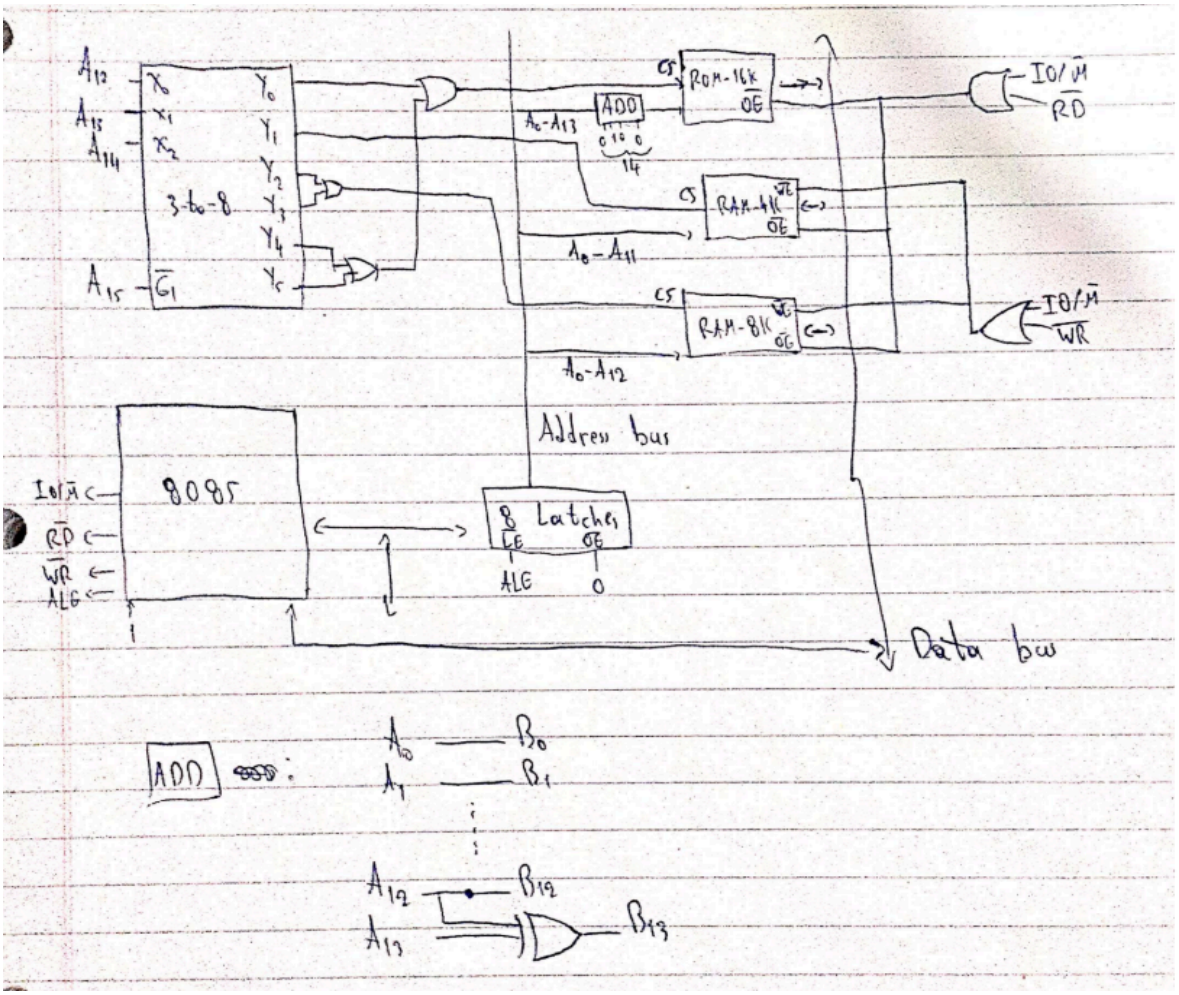


β) Χρήσει αποκλειστικά λογικών πυλών αντικαθιστώντας τον αποκωδικοποιητή:



7η Άσκηση

Ο χάρτης μνήμης είναι ο ακόλουθος:

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Address
Y_0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000
	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0FFF
Y_1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000
	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1FFF
Y_2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000
Y_3	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFF
Y_4	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000
Y_5	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	6FFF

και ο μΥ-Σ 8085 είναι ο παρακάτω: